The Pluggable CLP Connector for Presto Native: Specialized Storage for Semi-Structured Data

Article

- **Goal:** Explain CLP as specialized storage and show how Presto Native loads its connector without a fork.
- **Audience:** Presto users, operators, maintainers, and external connector authors.
- **Why:** Variant is becoming the lakehouse standard while Presto Native opens an out-of-tree connector model.
- **Takeaway:**
  - Choose CLP when upstream schemas and useful query paths cannot be controlled.
  - Recurring structure and static text interleaved with values improve compression and search locality.
  - The connector provides a practical reference for pluggable Presto Native connectors.

Opening: a common SQL type is not a storage design

- **Goal:** Establish the storage problem and historical transition.
- **Why:** Analytical platforms rarely control evolving upstream producers.
- **Takeaway:** Common SQL does not determine physical layout, compression, or search.
- **Important points:**
  - Schema tradeoff: JSON loses field-level execution. A global schema requires coordination.
  - YScope's August 2025 post owns the original CLP and optimizer walkthroughs.
  - Earlier integration: native execution in a Presto fork, requiring changes to Presto and Velox. Not a plugin.
  - Current connector: outside the Presto and Velox trees, loaded dynamically as a shared library.
  - Current availability: CLP works in PrestoDB through the connector. Variant does not yet work end to end.

1. Variant in the lakehouse landscape

- **Goal:** Separate Variant's SQL, table, and file layers.
- **Why:** Readers need the standard baseline before comparing CLP.
- **Takeaway:** Variant standardizes a dynamic value. Parquet determines its physical scan.
- **Important points:**
  - **Layers:** Trino as engine precedent, Iceberg v3 as table type, and Parquet as physical encoding.
  - **Unshredded:** Typed row-local value without independent inner-field columns or statistics.
  - **Shredded:** Selected paths gain typed columns, compression, and skipping.
  - Shredding choices are made before writing a file. Later changes require rewrites.
  - Exhaustive shredding creates a wide, sparse physical union.

2. CLP's distinctive storage model

- **Goal:** Retain only the CLP concepts required for the Variant comparison.
- **Why:** The August 2025 YScope post already covers the full storage and optimizer design.
- **Takeaway:** CLP is column-first and uses recurring structure and text to improve compression and narrow search.
- **Important points:**
  - **Parquet Variant:** One table per file with Variant values and optional shredded columns.
  - **CLP:** One archive containing homogeneous ERTs. ERTs are storage structures, not SQL tables.
  - The archive-local schema tree maps every observed path and type to physical columns.
  - **Sparsity:** Variant keeps one file-wide schema. ERTs omit absent fields but preserve explicit nulls.
  - **Logtypes:** Separate recurring static text from variables for compression and encoded search.
  - Compact archives can reduce object-storage transfer and request costs.
  - **Limits:** Stable schemas or nearly unique shapes reduce CLP's advantage.
  - Earlier conventional-Parquet benchmarks do not compare CLP against Variant.

3. From predicate to physical scan: Parquet Variant and CLP

- **Goal:** Compare reader work after pushdown without repeating optimizer mechanics.
- **Why:** The same SQL predicate can drive different physical scans.
- **Takeaway:** Parquet reads shredded columns or `value`. CLP resolves ERT columns once per archive split.
- **Important points:**
  - **Exact path:** Parquet uses a shredded column when available. CLP resolves matching ERT columns.
  - **Polymorphic path:** Parquet keeps non-declared types in `value`. CLP routes each type to homogeneous columns.
  - **Cross-field value:** Parquet inspects row-local `value` unless indexed. CLP uses archive metadata to select columns.
  - **Current CLP:** Typed value search over wildcard keys.

4. Beyond file pruning: how metastore planning and CLP archive metadata narrow the search

- **Goal:** Separate cross-file metadata from metadata inside an archive.
- **Why:** Each layer eliminates different work.
- **Takeaway:** Two metadata layers do more than prune splits.

- **Important points:**
  - **General planning:** Time bounds, dimensions, and statistics. Uber applies it across billions of file records.
  - **Top-N specialization:** Eligible newest-first queries can terminate before listing every file.
  - Correctness requires trustworthy time bounds, exact supported-predicate counts, and an overlap-aware watermark.
  - Time sorting and append-only publication help but are not required.
  - **Archive metadata:** Path/type map, logtype/value dictionaries, and indexes support streaming without local staging.
  - Uber's early termination is a production extension.
  - Structural/logtype summaries and semantic candidate narrowing remain future work.

5. From deployment to first query with the CLP connector

- **Goal:** Show the shortest operator path to a working catalog.
- **Why:** Readers should see the outcome before the implementation boundary.
- **Takeaway:** Ordinary Presto SQL drives a storage-native scan.
- **Important points:**
  - Setup: ingest with CLP, install matched artifacts, enable the loader, and configure `clp.properties`.
  - Default provider: MySQL metadata populated by CLP ingestion.
  - Current boundary: read-only access through the `default` schema.
  - Example query: typed paths, wildcard-key search, and whole-event reconstruction.
  - Compose examples: matched-artifact installation and complete deployment topology.

6. Building a pluggable Presto Native connector: CLP as a reference

- **Goal:** Explain the extension boundary and give connector authors a starting point.
- **Why:** This is the article's main Presto contribution.
- **Takeaway:** Native execution moved out of the fork, not out of the worker.
- **Important points:**
  - Ship the coordinator artifact, native bundle, and serialized protocol as one tested unit.
  - **Java:** Catalogs, metadata, splits, rewriting, and handle codecs.
  - **Native:** Load the `.so`, deserialize handles, scan CLP, and return columnar batches.
  - **Presto hooks:** PR #27653 for dynamic factories. PR #27652 for native deserialization.
  - Define tables, types, splits, pushdowns, residuals, and handles first.
  - Package the JAR, `.so`, dependencies, RPATH, and version pins together.
  - Match libc, architecture, Presto/Velox, CPU flags, and Folly `F14LinkCheck` mode.
  - Test Java planning with query runners. Use a real worker for runtime-resolved native code.

7. Conclusion: choosing a storage model for semi-structured data in Presto

- **Goal:** Convert the comparison into a workload decision.

- **Why:** Readers need conditions, not a universal winner.
- **Takeaway:** A common SQL interface can retain different physical scans.
- **Important points:**
  - **Choose Variant:** Cross-engine portability with predictable paths for shredding.
  - **Choose CLP:** Uncontrolled schemas, recurring structure/text, and cross-field or free-text search.
  - Stable schemas or nearly unique shapes reduce CLP's advantage.
  - **Current OSS:** Typed exact paths, typed wildcard-key value search, reconstruction, and pushdown across KV-IR files and archives.
  - **Uber:** About 25 PB/month raw to 750–850 TB. Five 48-core workers.
  - Uber's advanced planning is separate from current OSS.
  - Future Variant SQL could unify access without unifying storage.
  - Parquet would read `value` or shredded columns. CLP would bind paths to ERT columns and encoded streams.

Acknowledgments

- **YScope:** Jack Luo, Kirk Rodrigues, Rui Wang, Xiaochong Wei, and Chenxing Yang. Connector implementation.
- **IBM/Presto:** Pratik Joseph Dabre, Matt Graham, Deepak Majeti, and Tim Meehan. Infrastructure and review.
- **Uber:** Karthik Bhat, Qingqin Hou, Maksym Ivanchenko, and Yun Zhang. Production extensions and application with YScope.